

Mở rộng các bài toán quy hoạch động cơ bản

Yu Wei

Nguồn gốc: Further discussion of basic dynamic programming problems

IOI China candidate team papers / 2001 / Yu Wei.pdf

1 Mở đầu

Quy hoạch động có thể giải hiệu quả rất nhiều bài toán. Trong đó, nhiều mô hình toán học, đặc biệt là những mô hình được dùng cho các bài toán cơ bản đã được nghiên cứu từ năm 1957, đều rất tinh tế. Với các bài toán này, lời giải của chúng thậm chí có thể xem là chuẩn, tức là lời giải tối ưu.

Tuy nhiên, khi quy mô bài toán tăng lên, một số mô hình bộc lộ hạn chế và khuyết điểm của chính mình. Vì vậy ta cần tiếp tục tối ưu và cải tạo các mô hình đó.

2 Tối ưu ở mức chương trình

Tối ưu ở mức chương trình chủ yếu dựa vào tính đặc thù của bài toán. Xét phương trình truy hồi dạng

$$f(X_T) = \text{opt}\{f(u_T)\} + A(X_T), \quad u_T \in \text{Pred_Set}(X_T),$$

trong đó $A(X_T)$ là một hàm xác định theo X_T , còn $\text{Pred_Set}(X_T)$ là tập tiền nhiệm của X_T . Giả sử biến trạng thái X_T có số chiều là t , và mỗi X_T khác tiền nhiệm của nó ở e chiều. Từ phương trình trên, ta dễ dàng có một thuật toán thời gian $O(n^{t+e})$.

Tất nhiên thuật toán như vậy chưa phải tốt nhất. Để đơn giản hóa bài toán và thu được thuật toán tốt hơn, đặt

$$g(X_T) = \text{opt}\{f(u_T)\}.$$

Khi đó

$$f(X_T) = g(X_T) + A(X_T),$$

và vấn đề trở thành tính giá trị $g(X_T)$. Ta xét hai trường hợp.

2.1 Tập tiền nhiệm là tập liên tục

Trong trường hợp này, có thể dùng phương trình

$$g(X_T) = \text{opt}\{g(\text{Pred}(X_T)), f(\text{Pred}(X_T))\}$$

để tính $g(X_T)$, rồi dùng giá trị đó để tính $f(X_T)$. Như vậy, tuy về hình thức ta đã làm một quy hoạch động cho $g(X_T)$ và một quy hoạch động cho $f(X_T)$, hai quá trình này được tiến hành đồng thời, nên độ phức tạp giảm xuống $O(n')$.

Trong thực tế, các tập tiền nhiệm thường là liên tục, vì vậy phương pháp này có nhiều ứng dụng. Chẳng hạn bài “Little Shop of Flowers” của IOI 1999 có thể dùng cách này để giảm độ phức tạp nhìn bề ngoài từ $O(FV^2)$ xuống $O(FV)$.

2.2 Tập tiền nhiệm phụ thuộc vào trạng thái

Loại bài toán này phức tạp hơn. Lấy bài dãy con không giảm dài nhất làm ví dụ. Phương trình quy hoạch là

$$f(i) = \max\{f(j)\} + 1, \quad d[i] \geq d[j], \quad i > j.$$

Thông thường người ta cho rằng độ phức tạp khả thi thấp nhất của bài này là $O(n^2)$. Nhưng bài toán này chỉ có thêm một ràng buộc $d[i] \geq d[j]$; liệu có thể tối ưu tiếp hay không?

Xét phần $\max\{f(j)\}$, độ phức tạp trực tiếp của nó là $O(n)$. Với những biểu thức như vậy, ta thường có thể dùng hàng đợi ưu tiên hoặc cấu trúc dữ liệu có thứ tự để hạ phép lấy cực trị xuống $O(\log n)$. Với bài này cũng có thể làm như vậy.

Khi tính $d[i]$, trước hết dùng một cây nhị phân tìm kiếm cân bằng, chẳng hạn cây đỏ-đen, để sắp thứ tự $d[1], \dots, d[i-1]$. Với mỗi nút của cây, thêm một trường MAX ghi giá trị lớn nhất của hàm f trong cây con trái của nó. Khi tính $f(i)$, chỉ cần $O(\log n)$ để tìm nút ứng với số lớn nhất không vượt quá $d[i]$, rồi dùng $O(1)$ để truy cập trường MAX và nhận được giá trị $f(i)$. Các thao tác chèn và cập nhật trường MAX cũng chỉ mất $O(\log n)$; không cần thao tác xóa. Vì vậy tổng độ phức tạp là $O(n \log n)$.

Khi chạy thực tế, chương trình kiểu này cũng rất nhanh: với $n = 10000$, chưa tới một giây đã có kết quả, trong khi chương trình ban đầu cần khoảng 30 giây.

Từ thảo luận trên có thể thấy: khi tối ưu bài toán ở mức thiết kế chương trình, nên cố gắng giảm ràng buộc của bài toán và đưa nó về trường hợp tập tiền nhiệm liên tục. Nếu không thể, cần xem xét kỹ quan hệ trong dữ liệu và thiết kế cấu trúc dữ liệu phù hợp.

3 Tối ưu ở mức phương trình

Tối ưu ở mức phương trình chủ yếu dùng một số kết luận toán học để cải thiện phương trình và tránh những phép tính không cần thiết. Với một số bài toán đặc biệt, có thể phân tích toán học trực tiếp để tìm cực trị của phương trình, thậm chí không cần truy hồi giữa các trạng thái. Tuy nhiên số bài có thể giải bằng cách đó là hữu hạn, và phương pháp cũng khá phức tạp. Dù vậy, có khá nhiều bài toán tương đối tổng quát mà sau khi áp dụng một số kết luận toán học có thể nâng cao hiệu suất chương trình.

3.1 Cây nhị phân tìm kiếm tối ưu

Một ví dụ điển hình là bài cây nhị phân tìm kiếm tối ưu của CTSC 1996. Phương trình quy hoạch là

$$C[i, j] = w(i, j) + \min_{i < k \leq j} \{C[i, k-1] + C[k, j]\}, \quad 1 \leq i < j \leq n.$$

Từ phương trình này có ngay một thuật toán $O(n^3)$. Nhưng liệu có thuật toán hiệu quả hơn không?

Xét tính chất của $w(i, j)$. Nó biểu thị tổng tần suất của các nút từ i đến j . Rõ ràng nếu $[i, j] \subseteq [i', j']$, thì

$$w[i, j] \leq w[i', j'].$$

Từ đó biết được $C[i, j]$ có tính lồi theo bất đẳng thức Monge.

Để ký hiệu thuận tiện, đặt

$$C_k(i, j) = w(i, j) + C[i, k-1] + C[k, j],$$

và dùng $K_{i,j}$ để chỉ giá trị k làm $C_k(i, j)$ đạt tối ưu. Lấy $k = K_{i,j-1}$, và chọn $i < k' < k$. Vì $k' < k \leq j-1 < j$, ta có

$$C[k', j-1] + C[k, j] \leq C[k', j] + C[k, j-1].$$

Cộng hai vế với

$$w(i, j-1) + w(i, j) + C[i, k-1] + C[i, k'-1],$$

ta được

$$C_{k'}(i, j-1) + C_k(i, j) \leq C_{k'}(i, j) + C_k(i, j-1).$$

Theo định nghĩa của k , có

$$C_k(i, j-1) \leq C_{k'}(i, j-1),$$

nên $C_k(i, j) \leq C_{k'}(i, j)$. Do đó $k' \neq K_{i,j}$, suy ra

$$K_{i,j} \geq K_{i,j-1}.$$

Tương tự, có thể chứng minh

$$K_{i,j} \leq K_{i+1,j},$$

tức là

$$K_{i,j-1} \leq K_{i,j} \leq K_{i+1,j}.$$

Nhờ vậy ta có thể chia giai đoạn theo các đường chéo, tức theo $j-i$, để tính $K_{i,j}$. Thời gian tính một $K_{i,j}$ là

$$O(K_{i+1,j} - K_{i,j-1} + 1).$$

Ở giai đoạn thứ d , tức tính $K_{1,1+d}, \dots, K_{n-d,n}$, tổng thời gian không vượt quá $O(n)$. Có n giai đoạn, nên tổng độ phức tạp là $O(n^2)$.

Dù trong bài này hạn chế bộ nhớ làm thuật toán khó áp dụng trực tiếp, phương pháp đó vẫn đem lại gợi ý quan trọng.

3.2 Bài Post của IOI 2000

Xét bài POST của IOI 2000. Mô hình toán học của bài này không phải trọng tâm ở đây, nên ta bỏ qua phần dẫn xuất và viết trực tiếp phương trình:

$$D_{i,j} = \min_{i-1 \leq k < j} \{D_{i-1,k} + w(k,j)\}.$$

Thuật toán suy trực tiếp từ phương trình có độ phức tạp $O(n^3)$. Từ phương trình có thể thấy mỗi giai đoạn chỉ phụ thuộc vào giai đoạn trước. Vì vậy có thể đơn giản hóa thành việc thực hiện n lần phương trình

$$E[j] = \min_{j < i} \{D[i] + w(i,j)\}.$$

Trong truy hồi, giữa các giai đoạn không còn nhiều chỗ tối ưu, nên trọng tâm là tối ưu phương trình này. Dùng

$$B[i,j] = D[i] + w(i,j),$$

thuật toán ban đầu chính là tính B rồi lấy giá trị nhỏ nhất trên mỗi cột.

Điểm đặc biệt của bài là hàm w có tính chất sau. Với

$$a \leq b \leq c \leq d,$$

ta có

$$w(a, c) + w(b, d) \leq w(a, d) + w(b, c).$$

Tính chất này nên giúp được cho lời giải. Bắt chước ví dụ trên, cộng hai vế với $D[a] + D[b]$, ta được

$$B[a, c] + B[b, d] \leq B[a, d] + B[b, c].$$

Nói cách khác, nếu $B[a, c] \geq B[b, c]$, thì $B[a, d] \geq B[b, d]$. Vì vậy sau khi xác định quan hệ lớn nhỏ giữa $B[a, c]$ và $B[b, c]$, ta có thể quyết định có cần so sánh $B[a, d]$ và $B[b, d]$ nữa hay không.

Xa hơn, chỉ cần tìm giá trị nhỏ nhất h sao cho

$$B[a, h] \geq B[b, h],$$

thì có thể bỏ qua việc tính cột a sau vị trí h . Giá trị h này có thể tìm bằng nhị phân trong $O(\log n)$. Nếu w đặc biệt hơn, chẳng hạn là một hàm xác định đủ đơn giản, thậm chí có thể tìm h trong $O(1)$. Với mỗi hàng chỉ cần thực hiện một lần tìm kiếm nhị phân. Sau khi tìm tất cả các h , chỉ cần thêm $O(n)$ thời gian để lấy giá trị cần thiết theo từng cột. Tổng thời gian là

$$O(n) + O(n \log n) = O(n \log n).$$

Các chi tiết cài đặt không quá phức tạp nhưng không phải trọng tâm. Do bài này còn có thể dùng kỹ thuật mảng cuộn để giải quyết bộ nhớ, thuật toán này có biểu hiện rất tốt trên dữ liệu lớn.

Từ mô tả trên có thể thấy, tối ưu phương trình chủ yếu phụ thuộc vào tính chất của hàm trọng số w . Bất đẳng thức

$$w(a, c) + w(b, d) \leq w(a, d) + w(b, c)$$

là loại được dùng nhiều nhất; thực chất nó là bất đẳng thức xác định tính lồi của hàm. Trong các bài toán thực tế, hàm trọng số thường thỏa bất đẳng thức này hoặc bất đẳng thức ngược lại, nên kiểu tối ưu này có phạm vi ứng dụng khá rộng. Ngoài ra còn nhiều bất đẳng thức đặc biệt khác; nếu áp dụng được trong chương trình, chúng đều có thể nâng cao hiệu suất.

4 Chuyển từ thấp chiều sang cao chiều

Khi mở rộng quy mô bài toán, một cách mở rộng là tăng số chiều của bài. Lúc này số chiều của biến quyết định trong quy hoạch cũng tăng. Không gian lưu trữ vì vậy tăng theo cấp số nhân, dẫn đến không thể lưu toàn bộ trạng thái. Đây là vấn đề “tai họa số chiều” của quy hoạch động.

Nếu vẫn muốn dùng quy hoạch động trong tình huống này, thường phải dùng phân tích toán học rất phức tạp. Với chúng ta, cách đó rõ ràng không thực tế. Khi ấy cần cải tạo mô hình quy hoạch động. Thông thường, ta có thể biến mô hình quy hoạch động lúc này thành mô hình luồng mạng.

Về cách chuyển mô hình, có một số quy luật tổng quát. Nếu phương trình chuyển trạng thái chỉ liên quan đến một trạng thái khác, ta có thể chắc chắn thu được một mô hình luồng cực đại chi phí nhỏ nhất. Mô hình này tất nhiên cũng có quy luật riêng; thậm chí khi dùng thuật toán đối ngẫu để giải luồng mạng, vẫn có thể phải dùng tư tưởng quy hoạch động. Nhưng đó không phải trọng tâm ở đây; điều ta quan tâm là cách chuyển một bài toán quy hoạch động.

Ví dụ, bài “Mars Explorer” của IOI 1997 có mô hình một chiều có thể giải bằng quy hoạch động, trong đó khái niệm chiều ở đây chỉ số lượng tàu thăm dò. Khi số chiều tăng lên, có thể dùng cách nói trên để chuyển sang luồng mạng.

Ngoài ra còn nhiều bài khác có thể giải bằng phương pháp này. Chẳng hạn bài phủ đoạn dài nhất, khi số chiều tăng lên, cũng có thể xử lý tương tự. Xa hơn, ngay cả bài đường đi ngắn nhất trong đồ thị cũng có thể chuyển hóa kiểu này để tìm một số đường đi ngắn nhất đặc biệt.

Tuy nhiên, nói chung sau khi chuyển hóa, lưu lượng cực đại thường là 1; nhiều tính chất đặc biệt chưa được khai thác. Một số bài quy hoạch động phức tạp cũng chưa thể chuyển thành luồng mạng, chẳng hạn bài cây nhị phân tối ưu. Vì vậy dùng thuật toán luồng mạng chuẩn đôi khi là lãng phí, và cách giải các trường hợp này còn cần nghiên cứu thêm.

Tài liệu tham khảo

References

- [1] David Eppstein, Zvi Galil và Raffaele Giancarlo. *Speeding up Dynamic Programming*.
- [2] Zvi Galil và Kunsoo Park. *Dynamic Programming with Convexity, Concavity, and Sparsity*.

Phụ lục

Tính lỗi của $C[i, j]$. Tính lỗi ở đây nghĩa là với mọi $a \leq b \leq c \leq d$,

$$C[a, c] + C[b, d] \leq C[a, d] + C[b, c].$$

Một chứng minh phác thảo như sau. Đặt $k = K_{b,c}$, khi đó

$$\begin{aligned} C[a, c] + C[b, d] &\leq C[a, k-1] + C[k, c] + C[b, k-1] + C[k, d] + w(a, c) + w(b, d) \\ &= C[a, k-1] + C[k, d] + w(a, d) + C[b, k-1] + C[k, c] + w(b, c) \\ &= C[a, d] + C[b, c]. \end{aligned}$$

Vậy bất đẳng thức được chứng minh.

Giải mã cho tối ưu bài Post.

```
begin
  E[1] <- D[1]
  Queue.Add(K:1, H:n)
  for j <- 2 to n do
    begin
      if B(j-1, j) <= B(Queue.K[head], j) then
        begin
          E[j] <- B(j-1, j)
          Queue.Empty
          Queue.Add(K:j-1, H:n+1)
        end
      else
        begin
          E[j] <- B(Queue.K[head], j)
          while B(j-1, Queue.H[tail-1])
            <= B(Queue.K[tail], Queue.H[tail-1]) do
            Queue.Delete(tail)
          Queue.H[tail] <- h(Queue.K[tail], j-1)
          if h_0K then Queue.Add(K:j-1, H:n+1)
        end
      if Queue.H[head] = j+1 then Queue.SkipHead
    end
  end
```

Hàng đợi Queue có thể gọi là hàng đợi ứng viên; đầu hàng đợi là giá trị nhỏ nhất của hàng thứ j , và giả sử $\text{Queue.H}[0] = j$. Hàm $h(a, b)$ dùng tìm kiếm nhị phân để tìm giá trị nhỏ nhất h sao cho

$$B(a, h) \geq B(b, h),$$

còn h_{OK} cho biết việc tìm kiếm có thành công hay không. Trong hàng đợi ứng viên, dữ liệu dạng $(K : i_r, H : h_r)$ nghĩa là: khi chỉ số hàng nằm trong đoạn tương ứng, cột i_r là nơi đạt giá trị nhỏ nhất.

Chuyển quy hoạch động sang luồng chi phí nhỏ nhất. Quy hoạch động thực chất là một cách tìm đường đi ngắn nhất trên đồ thị có hướng. Vì vậy có thể xem mỗi trạng thái là một đỉnh, và quá trình chuyển trạng thái là các cạnh của đồ thị. Khi chuyển sang mô hình luồng cực đại chi phí nhỏ nhất, tách mỗi đỉnh thành một đỉnh vào và một đỉnh ra. Các cạnh đi vào đỉnh cũ nối tới đỉnh vào; các cạnh đi ra từ đỉnh cũ xuất phát từ đỉnh ra. Các cạnh cũ có dung lượng dương vô hạn, trọng số thường giữ nguyên. Nối thêm một cạnh từ đỉnh vào sang đỉnh ra, trọng số bằng trọng số của đỉnh, dung lượng bằng số lần mỗi đỉnh được phép đi qua, thường là một.

Sau đó tạo siêu nguồn và siêu đích, nối chúng với các đỉnh vào hoặc đỉnh ra có thể tương ứng. Nếu cần, siêu nguồn hoặc siêu đích cũng phải tách thành hai đỉnh, và cạnh giữa hai đỉnh đó có dung lượng bằng dung lượng lớn nhất có thể, trọng số bằng 0. Khi đó nghiệm thu được bằng luồng chi phí nhỏ nhất chính là nghiệm của bài toán trong trường hợp nhiều chiều.

Các chương trình nguồn kèm theo bản gốc. PDF gốc liệt kê các tệp chương trình cho dãy con không giảm dài nhất, cây nhị phân tìm kiếm tối ưu và bài Post, gồm các bản thường, bản cải tiến và chương trình sinh dữ liệu: Lennor.pas, Lentree.pas, Lengern.pas, Btreenor.pas, Btreespe.pas, Tgern.pas, post.pas, Pbig.pas, Pspe.pas, pgern.pas, Pbgern.pas.